Stable Matching

CMSC 27200: Theory of Algorithms

January 6, 2025

Corresponding section(s) of Kleinberg-Tardos: 1.1

1 Stable matching

Problem 1.1 (Stable matching)**.** *In the stable matching problem, we have the following data:*

1. *There are two sets of n people, $A = \{a_1, \ldots, a_n\}$ and $B = \{b_1, \ldots, b_n\}$.*
2. *Each person $a_i \in A$ has a preference order on the people $b_1, \ldots, b_n \in B$.*
3. *Each person $b_j \in B$ has a preference order on the people $a_1, \ldots, a_n \in A$.*

We are then asked to find a stable matching M between A and B, which is a matching M such that there does not exist a pair of people a_i and b_j who would both rather be with each other than with their partners in M.

We can make this more precise with the following definitions.

Definition 1.2 (Unstable pair)**.** *Given the data above and a matching $M = \{(a_i, b_{\pi(i)}) : i \in [n]\}$ where $\pi : [n] \to [n]$ is a permutation (i.e., a bijective map from $[n]$ to $[n]$), an **unstable pair** is a pair (a_i, b_j) such that $i, j \in [n]$ and*

1. *$\pi(i) \neq j$ (a_i and b_j are not currently partners).*
2. *a_i prefers b_j to $b_{\pi(i)}$ (a_i prefers b_j to his/her current partner).*
3. *b_j prefers a_i to $a_{\pi^{-1}(j)}$ (b_j prefers a_i to his/her current partner).*

Definition 1.3 (Stable matching)**.** *Given the above data, a **stable matching** is a matching $M = \{(a_i, b_{\pi(i)}) : i \in [n]\}$ where $\pi : [n] \to [n]$ is a permutation and M has no **unstable pairs**.*

With these definitions, the stable matching problem can be restated as follows. Given the above data, find a **stable matching**.

Example 1.4. *If the preference orders for group A are*

$a_1 : b_1, b_2, b_3$

$a_2 : b_1, b_3, b_2$

$a_3 : b_3, b_2, b_1$

and the preference orders for the people in group B are

$b_1 : a_3, a_1, a_2$

$b_2 : a_2, a_1, a_3$

$b_3 : a_2, a_3, a_1$

*then the matching $(a_1, b_1), (a_2, b_2), (a_3, b_3)$ is unstable because (a_2, b_3) is an **unstable pair**. The two **stable matchings** are $(a_1, b_1), (a_2, b_3), (a_3, b_2)$ and $(a_1, b_2), (a_2, b_3), (a_3, b_1)$.*

Algorithm 1.5 (Gale-Shapley algorithm for stable matching)**.** *In the Gale-Shapley Algorithm, the people in group A go down their preference lists and ask the people in group B if they would like to be partners. When a person b_j in group B receives such an offer from a person a_i in A, b_j accepts the offer if he/she currently has no partner or has a partner $a_{i'}$ but prefers a_i to $a_{i'}$ (in which case $a_{i'}$ becomes unmatched and must start making offers again starting from the next person down in his/her preference list). Otherwise, b_j turns the offer down.*

More precisely, the Gale-Shapley algorithm is as follows

Stored data: In the Gale-Shapley algorithm, we keep track of a map $\pi : [n] \to [n] \cup \{0\}$ which describes the current partnerships between A and B. If $\pi(i) = j$ for some $j \in [n]$ then a_i is currently partners with b_j and if $\pi(i) = 0$ then i currently has no partner. Thus, π corresponds to the partial matching $M = \{(a_i, b_{\pi(i)}) : i \in [n], \pi(i) \neq 0\}$

Inititialization: We start with $\pi(i) = 0$ for all $i \in [n]$. In other words, we start with no partnerships between A and B.

Iterative step: At each step, we choose an i such that $\pi(i) = 0$ (i.e., a_i is unmatched) and have a_i ask person b_j if he/she would like to be partners where b_j is a_i's most preferred person that a_i has not yet asked. b_j then responds as follows

1. *If $\nexists i' \in [n]\,(\pi(i') = j)$ (i.e., b_j is currently unmatched) then we set $\pi(i) = j$, making a_i and b_j partners.*
2. *If $\exists i' \in [n]\,(\pi(i') = j)$ (i.e., b_j is currently matched with $a_{i'}$) and b_j prefers a_i to $a_{i'}$ then we set $\pi(i) = j$ and set $\pi(i') = 0$. This breaks the partnership between $a_{i'}$ and b_j and makes a_i and b_j partners.*
3. *If $\exists i' \in [n]\,(\pi(i') = j)$ (i.e., b_j is currently matched with $a_{i'}$) and b_j prefers $a_{i'}$ to a_i then b_j turns down a_i's offer and we keep π as is.*

Once everyone in group A is matched or has nobody left to make an offer to, the algorithm terminates.

Note that the sequence of offers which is made is not fixed. That said, we will show that as long as each person in group A makes offers according to their preference list, the final matching will be the same regardless of who makes their offer at each step. In fact, the final matching will be the best possible stable matching for group A and the worst possible stable matching for group B.

If we instead have group B make the offers then we may obtain a different stable matching. In fact, we will obtain the matching which is the best possible stable matching for group B and the worst possible stable matching for group A.

Example 1.6. *If we have the following preference lists*

Group A's preference lists (from most preferred to least preferred):

a_1: b_1, b_3, b_2

a_2: b_1, b_2, b_3

a_3: b_3, b_2, b_1

Group B's preference lists (from most preferred to least preferred):

b_1: a_3, a_2, a_1

b_2: a_2, a_3, a_1

b_3: a_2, a_1, a_3

then one possible sequence of offers for the Gale-Shapley algorithm (with group A making the offers) is as follows. We start with $M = \emptyset$.

1. *a_1 makes an offer to b_1 which is accepted as b_1 is unmatched. Resulting matching: $M = \{(a_1, b_1)\}$.*
2. *a_2 makes an offer to b_1 which is accepted as b_1 prefers a_2 to a_1. Resulting matching: $M = \{(a_2, b_1)\}$.*
3. *a_1 makes an offer to b_3 which is accepted as b_3 is unmatched. Resulting matching: $M = \{(a_1, b_3), (a_2, b_1)\}$.*
4. *a_3 makes an offer to b_3 which is rejected as b_3 prefers a_1 to a_3. Resulting matching: $M = \{(a_1, b_3), (a_2, b_1)\}$.*
5. *a_3 makes an offer to b_2 which is accepted as b_2 is unmatched. Final matching: $M = \{(a_1, b_3), (a_2, b_1), (b_2, a_3)\}$.*

If we instead had group B making the offers then one possible sequence of offers is as follows:

1. *b_1 makes an offer to a_3 which is accepted as a_3 is unmatched. Resulting matching: $M = \{(a_3, b_1)\}$.*
2. *b_2 makes an offer to a_2 which is accepted as a_2 is unmatched. Resulting matching: $M = \{(a_3, b_1), (a_2, b_2)\}$.*

3. *b_3 makes an offer to a_2 which is rejected as a_2 prefers b_2 to b_3. Resulting matching: $M = \{(a_3, b_1), (a_2, b_2)\}$.*

4. *b_3 makes an offer to a_1 which is accepted as a_1 is unmatched. Final matching: $M = \{(a_1, b_3), (a_3, b_1), (a_2, b_2)\}$.*

1.1 Analysis of the Gale-Shapley algorithm

Theorem 1.7. *The Gale-Shapley algorithm always terminates in at most n^2 iterations and results in a stable matching.*

Proof. The intuition behind the Gale-Shapley algorithm is as follows. For the people in group B, their partner only gets better with time as they only accept a new partnership if it is better than their current partnership (if any). Thus, once a person a_i in group A is turned down by a person b_j in group B, either because a_i's initial offer was turned down or because b_j left a_i to partner with another person $a_{i'}$, this rejection is final; b_j will never regret turning down a_i.

This means that when the algorithm finishes, all of the people in A are as happy as they can reasonably be. There might be people in group B that they'd rather be with, but those people are unattainable. This makes the resulting matching stable.

We now turn this intuition into a proof. For this, the following definition is useful (this differs slightly from the presentation in Kleinberg-Tardos):

Definition 1.8. *We say that b_j is available to a_i if any of the following cases hold:*

1. *$\pi(i) = j$ (b_j is already matched to a_i)*

2. *$\nexists i'(\pi(i') = j)$ (b_j is unmatched)*

3. *$\exists i'(\pi(i') = j)$ but b_j prefers a_i to $a_{i'}$ (b_j is currently matched but would switch to partner with a_i if asked)*

Otherwise, we say that b_j is unavailable to a_i.

When analyzing algorithms, it is often useful to find invariants which remain true as the algorithm progresses. Here we have the following invariants:

Lemma 1.9. *As the algorithm progresses,*

1. *Every person $b_j \in B$ only gets happier. In other words, once a person $b_j \in B$ has a partner they will never be unmatched again and whenever b_j changes partners, b_j is happier with his/her new partner.*

2. *For all $i, j \in [n]$, if b_j becomes unavailable to a_i then b_j never becomes available to a_i again.*

Proof. For the first statement, observe that once b_j has a partner a_i, the only way this partnership will break is if b_j accepts an offer from another person $a_{i'} \in A$ in which case b_j remains matched and is happier with his/her new partner.

To see the second statement, assume that b_j becomes unavailable to a_i and then later becomes available to a_i. Let $a_{i'}$ be b_j's partner when b_j becomes unavailable to a_i and let $a_{i''}$ be b_j's partner when b_j becomes available again. We must have that b_j prefers $a_{i'}$ to a_i and b_j prefers a_i to $a_{i''}$

which implies that b_j prefers $a_{i'}$ to $a_{i''}$. However, by the first statement, this is impossible as b_j only gets happier as time goes on. □

Remark 1.10. *This is a common strategy for proving that something can never happen. We assume that it does happen and then derive a contradiction.*

With these invariants in hand, we now prove that the Gale-Shapley algorithm always terminates in at most n^2 iterations and when it does, it results in a stable matching. To see that the Gale-Shapley algorithm must terminate in at most n^2 iterations, note that in each step, a person from group A makes an offer to a person from group B which they have not yet made an offer to. This can happen at most n^2 times, so there can be at most n^2 iterations.

Remark 1.11. *Here the number of proposals made by people in group A is a measure of progress which allows us to see that the algorithm is making progress and will eventually terminate.*

To see that the Gale-Shapley algorithm results in a matching, note that the only way the Gale-Shapley algorithm can fail to give a matching is if some person a_i in group A is rejected by every person in group B. However, in order for this to happen, every person in group B must be partnered with someone who they prefer to a_i. However, this is impossible because there are n people in group B and there are only $n-1$ other people in group A.

Finally, to see that the Gale-Shapley algorithm results in a stable matching, assume that the resulting map/matching is not stable, i.e. there exist $i, j \in [n]$ such that

1. $\pi(i) \neq j$
2. a_i prefers b_j to $b_{\pi(i)}$
3. b_j prefers a_i to $a_{\pi^{-1}(j)}$

If so, then on the one hand b_j is available to a_i because b_j prefers a_i to $a_{\pi^{-1}(j)}$. However, on the other hand, since a_i prefers b_j to $b_{\pi(i)}$, a_i would have first made an offer to b_j before making an offer to $b_{\pi(i)}$. Since a_i is not with b_j, a_i must have previously made an offer to b_j and been turned down, either initially or because b_j received a better offer. At this point, b_j was unavailable to a_i so by Lemma 1.9, b_j must still be unavailable to a_i, which is a contradiction. □

In fact, we can say more about the Gale-Shapley algorithm. When we gave intuition for the algorithm, we claimed that the people in group A will be as happy as they can reasonably be. This statement can be made precise and proved.

Theorem 1.12. *No matter which order we have the people in group A make their offers, the resulting map/matching π/M will be the same. Moreover, this stable matching is the best possible stable matching for the people in group A and the worst possible stable matching for the people in group B. More precisely, for any other stable map/matching π'/M',*

1. *For all $i \in [n]$, either $\pi'(i) = \pi(i)$ or a_i prefers $b_{\pi(i)}$ to $b_{\pi'(i)}$*
2. *For all $j \in [n]$, either $\pi'^{-1}(j) = \pi^{-1}(j)$ or b_j prefers $a_{\pi'^{-1}(j)}$ to $a_{\pi^{-1}(j)}$*

Proof.

Definition 1.13. *For each $i \in [n]$, define S_i to be the set of possible partners for a_i in a stable matching, i.e.*

$$S_i = \{j : \textit{ There exists a stable map/matching } \pi'/M' \textit{ such that } \pi'(i) = j\}$$

Lemma 1.14. *Whenever we run the Gale-Shapley algorithm, there is never an $i, j \in [n]$ such that*

1. *$j \in S_i$*
2. *b_j is unavailable to a_i.*

Proof. Assume that this does happen. If so, consider the first time where there is an $i, j \in [n]$ such that $j \in S_i$ but b_j is unavailable to a_i. At this point, b_j must have just accepted an offer from $a_{i'}$ for some $i' \in [n]$.

Now note that since $j \in S_i$, there must be some stable map/matching π/M such that $\pi(i) = j$. Let $j' = \pi(i')$. We now consider whether $a_{i'}$ prefers b_j or $b_{j'}$. On the one hand, if $a_{i'}$ prefers b_j to $b_{j'}$ then since b_j prefers $a_{i'}$ to a_i, π/M is not a stable matching, which is a contradiction. On the other hand, if $a_{i'}$ prefers $b_{j'}$ to b_j then before making an offer to b_j, $a_{i'}$ must have made an offer to $b_{j'}$ and been turned down or turned away. Since $j' \in S_{i'}$, this is an earlier case where $i', j' \in [n]$, $j' \in S_{i'}$, and $b_{j'}$ is unavailable to $a_{i'}$, which contradicts the definition of i and j. □

Corollary 1.15. *If π/M is a stable map/matching resulting from the Gale-Shapley algorithm then for all $i \in [n]$, if $\pi(i) = j$ then*

1. *$j \in S_i$*
2. *For all $j' \in S_i$ such that $j' \neq j$, a_i prefers b_j to $b_{j'}$.*

Proof. The first statement follows immediately from the fact that the Gale-Shapley algorithm gives a stable matching. For the second statement, note that by the lemma we just proved, whenever we run the Gale-Shapley algorithm, for all $j' \in S_i$, $b_{j'}$ is always available to a_i. Thus, a_i never needs to make an offer to anyone below their top choice in S_i. □

We can now prove Theorem 1.12. Let π/M is a stable map/matching resulting from the Gale-Shapley algorithm and let π'/M' be a different stable matching. Corollary 1.15 implies that M must be the matching $\{(i, a_i\text{'s most preferred partner in } S_i) : i \in [n]\}$. We just need to show that whenever $\pi'^{-1}(j) \neq \pi^{-1}(j)$, b_j prefers $a_{\pi'^{-1}(j)}$ to $a_{\pi^{-1}(j)}$. To see this, observe that $a_{\pi^{-1}(j)}$ must prefer b_j to $b_{\pi'\pi^{-1}(j)}$ because both j and $\pi'\pi^{-1}(j)$ are in $S_{\pi^{-1}(j)}$ and $a_{\pi^{-1}(j)}$ gets his/her most preferred partner in $S_{\pi^{-1}(j)}$ in the map/matching π/M. Thus, b_j must prefer $a_{\pi'^{-1}(j)}$ to $a_{\pi^{-1}(j)}$ because otherwise both $a_{\pi^{-1}(j)}$ and b_j would prefer each other to their current partners in M' so M' would not be stable. □

1.2 Data structures for implementing Gale-Shapley

To implement Gale-Shapley, we can use the following data structures.

1. The preference lists for group A and the preference lists for group B.

2. The number of people m who are already matched and the index i such that a_i will make the next offer. If we have each person in group A make a new offer whenever they are rejected or lose their partner, we will have that until the algorithm terminates, all of the people $\{a_1, \ldots, a_{m+1}\} \setminus \{a_i\}$ are matched and the other people in group A are unmatched.

3. An array keeping track of how far down each person in group A is in their preference lists. This data (along with i) allows us to determine the next offer which is made. From this data, we can derive the map $\pi : [n] \to [n] \cup \{0\}$ where $\pi(i) = j$ if a_i is matched with b_j and $\pi(i) = 0$ if a_i is unmatched.

4. A map $\pi^* : [n] \cup \{0\}$ (which will become π^{-1} when the algorithm terminates) where $\pi^*(j) = i$ if b_j is matched with a_i and $\pi^*(j) = 0$ if b_j is unmatched.

5. An $n \times n$ matrix P where P_{ij} is the rank of a_i in b_j's preference list. In other words, $P_{ij} = k$ if a_i is the kth element of b_j's preference list. This auxiliary data structure can be constructed from group B's preference lists and is needed (along with π^*) in order to quickly determine whether the people in group B accept or reject the offers they receive.

With these data structures, each iteration can be implemented in constant time so the total runtime for Gale-Shapley is $O(n^2)$. For more details, see Section 2.3 of Kleinberg-Tardos.